

Labor Kryptologie, Blatt 5

Johannes Bauer & Reinhold Hübl & Miriam Weigel *

Sommersemester 2025

Allgemeiner Hinweis: Wenn nicht explizit anders spezifiziert hat Ihre Abgabe als PDF zu erfolgen, das grundsätzlich handschriftlich ausgeführt ist. Die Ausnahme stellt Programmcode dar, welchen Sie allerdings immer als separate Datei mit abgeben sollen. Zulässige Programmiersprachen sind C, C++, Python, Java, Rust, Go.

1 Feistel-Chiffren

Gegeben sei eine balancierte Feistel-Chiffre „Schiffy-128“ mit Blockgröße und Schlüssellänge von je 128 Bit. Schiffy-128 verwendet eine 8×8 S-Box, welche wie folgt definiert ist:

$$S_0 = 170$$

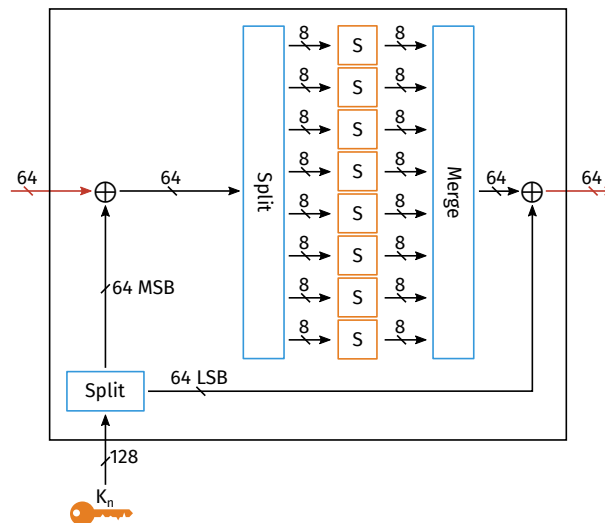
$$S_x = 37 \cdot S_{x-1} + 9 \pmod{256}$$

Schiffy-128 verwendet 32 Runden und hat entsprechend auch 32 Rundenschlüssel, die jeweils 128 Bit lang sind. Ausgehend von einem Schlüssel K werden innerhalb des KSA die 32 Rundenschlüssel $K_0 \dots K_{31}$ wie folgt abgeleitet:

$$K_{-1} = K$$

$$K_x = (K_{x-1} \lll 7x) \oplus 0xabcdef$$

Die Feistel-Funktion F von Schiffy-128 nimmt einen 64-Bit Block und einen 128-Bit Rundenschlüssel als Eingaben und produziert einen 64-Bit Block als Ausgabe. Schematisch sieht F wie folgt aus:



*johannes.bauer@dhbw.de reinhold.huebl@dhbw.de; miriam.weigel@dhbw.de; DHBW Mannheim

Konkret wird der Rundenschlüssel K_n also in zwei Teile geschnitten, die höherwertigen (MSB) 64 Bit sowie die niederwertigen (LSB) 64 Bit. Auf die Eingabedaten werden zunächst die höherwertigen 64 Bit des Rundenschlüssels per XOR addiert. Dann wird jeweils byteweise die Schiffy S-Box appliziert. Im letzten Schritt werden auf den Wert nochmals die niederwertigen 64 Bit des Rundenschlüssels per XOR addiert.

Die Feistel-Konstruktion von Schiffy-128 verhält sich wie in der Vorlesung gezeigt.

Folgende Testvektoren zur S-Box seien Ihnen gegeben:

$$S_0 = 170$$

$$S_1 = 155$$

$$S_2 = 112$$

$$S_{123} = 33$$

$$S_{255} = 205$$

Unter dem Schlüssel K

$$K = \text{deadbeef0000000000000000badc0ffee}$$

Ergeben sich folgende exemplarischen Rundenschlüssel:

$$K_0 = \text{deadbeef0000000000000000bad6b3201}$$

$$K_1 = \text{56df77800000000000000005d6b532cd00}$$

$$K_2 = \text{dde00000000000000000175ad4cb3ebd858}$$

$$K_{31} = \text{770feb4b3180dc3bc09870bd38e2cb5f}$$

Der Nullblock, verschlüsselt unter K , hat folgenden Ciphertext:

$$E(00000000000000000000000000000000, K) = \text{b743f2fb342c51bfab950797083f61e9}$$

Bei dieser Verschlüsselung werden jeweils folgende Evaluationen von F verwendet:

$$F(0000000000000000, K_0) = \text{94dfb49607c198ab}$$

$$F(94dfb49607c198ab, K_1) = \text{b0aa7cca50e95fb1}$$

$$F(\text{b0aa7cca50e95fb1}, K_2) = \text{1e9d6324e9783573}$$

$$F(8a42d7b2eeb9add8, K_3) = \text{01a6283b0f33c8f0}$$

$$F(\text{c8ef99ba72f8a579}, K_{29}) = \text{f7ffea032144154a}$$

$$F(81f3d4d01743d570, K_{30}) = \text{7fac6b4146d4f4c6}$$

$$F(\text{b743f2fb342c51bf}, K_{31}) = \text{2a66d3471f7cb499}$$

Ihnen ist auch unter dem Schlüssel $Q = \text{08150000000000000000000000004711}$ ein ECB-verschlüsselter Ciphertext bekannt, dessen Plaintext Sie allerdings nicht kennen und den Sie als unbenotete Bonusaufgabe entschlüsseln könnten:

$$C = \text{2aed234f7ceda0ba9a89118bc0a0b93fe5b820aac165b97e3ad6338d23cb5858}$$

1. (5 Punkte) Implementieren Sie Schiffy-128. Geben Sie allen kommentierten Programmcode als ZIP-Datei namens „solution.zip“ ab. Achten Sie darauf, dass der Name der ZIP-Datei **exakt** stimmt. Plausibilisieren Sie in Ihrem Programmcode die Testvektoren **automatisiert** (z.B. mittels Assertions).
2. (3 Punkte) Verschlüsseln Sie mit Ihrem Programm eine möglichst originelle Nachricht mittels CBC unter dem Schlüssel Q und geben diese als Binärdatei „ciphertext.bin“ ab. Der Dateiname muss **exakt** stimmen und die Größe der Datei muss auch unbedingt ein Vielfaches von 16 Bytes sein. Die ersten 16 Bytes der Datei müssen ein von Ihnen **randomisiert** gewählter IV sein.